BMM Developer Notes

Table of contents

Overview 4
- From idea to merged model . . . 4
- How to read these notes . . . 4
- The rules for contributing . . . 5

1 System setup 6
- 1.1 Setting up a local fork of the bmm repository . . . 6
- 1.2 Prerequisites . . . 6
- 1.3 Package development with devtools . . . 7
- 1.4 Running the tests . . . 7
- 1.5 Untracked scratch work . . . 8

2 Git and GitHub workflow 9
- 2.1 Branches . . . 9
- 2.2 Git commands . . . 11
- 2.3 Pull requests . . . 11

3 BMM code structure 12
- 3.1 What happens when a user calls `bmm()` . . . 12
- 3.2 The `bmmformula` class . . . 13
- 3.3 Models . . . 13
- 3.4 Class chains . . . 15
- 3.5 S3 generics . . . 16
- 3.6 Passing information from `check_data()` to `configure_model()` . . . 17
- 3.7 Re-running `check_data()` in `update()` . . . 17
- 3.8 Default priors . . . 18
- 3.9 File organization . . . 18

4 Example models 21
- 4.1 The Interference Measurement Model (IMM) . . . 21
  - 4.1.1 Model definition . . . 21
  - 4.1.2 `check_data()` and the dispatch chain . . . 21
  - 4.1.3 `configure_model()` . . . 22
  - 4.1.4 From `bmmformula` to `brmsformula` . . . 22
- 4.2 The Diffusion Decision Model (DDM) . . . 22
  - 4.2.1 Model definition . . . 23

4.2.2 `bmf2bf()` . . . 23
4.2.3 `configure_model()` . . . 23

5 Adding a new model **32**
What `use_model_template()` does . . . 32
What the template leaves for you . . . 32
5.1 Example . . . 33
5.2 Filling in the template . . . 35
5.3 Testing . . . 37
5.3.1 Parameter recovery . . . 39
5.3.2 Distribution functions . . . 39
5.4 Before you open the pull request . . . 39
5.5 Failure modes to expect . . . 40
5.6 Post-processing . . . 41
5.7 Add an example dataset . . . 41
5.8 Add an article . . . 41

6 Coding conventions **42**
6.1 Errors, warnings and messages . . . 42
6.2 Dependencies . . . 43
6.3 Code shape . . . 44
6.4 S3 conventions . . . 45
6.5 Where to validate arguments . . . 45
6.6 `log_lik`, `posterior_predict` and `posterior_epred` . . . 46
6.7 Documentation and the checks that enforce it . . . 47
6.8 Tests . . . 48
6.9 Parameter recovery: the circularity trap . . . 48

Appendices **49**

A Reference **49**
A.1 Links . . . 49
A.2 Models without a location parameter . . . 49
A.3 Initial values . . . 50
A.4 Aggregated data and `vars` . . . 51
A.5 Within-chain threading . . . 51
A.6 Version tables . . . 52
A.7 Choosing the `sd` prior rate . . . 53
A.8 Naming traps . . . 54

Overview

Last update: 16.10.2026

These notes are for developers who want to add a model to bmm. They explain how the package is put together and walk through the steps of contributing a model, from setting up your system to opening the pull request.

bmm covers five kinds of tasks: continuous reproduction (mixture models, IMM, SDM), categorical recall and n-AFC decisions (M3), processing-tree models (MPT), detection, recognition and confidence judgments (the signal detection models), and choices with response times (DDM, EZDM, Censored-Shifted Wald). `bmm_models()` lists them. The package requires **R >= 4.1.0** and uses the native pipe `|>` throughout.

This guide describes bmm at the **v1.4.1** tag. Anything merged after that tag may not be reflected here yet. If you run into problems, open an issue on GitHub.

From idea to merged model

1. Open an issue with the `new_model` template and agree with the maintainers on the model and the release it targets.
2. Fork bmm, set up your system and create a branch named after the issue (Chapter 1).
3. Generate the model file with `use_model_template()` and fill it in (Chapter 5). Write the distribution functions first and the prediction functions after.
4. Add the model's tests and its test cases, and run `document()`, `test()` and `check()`.
5. Run a parameter recovery study, and add an example dataset, an article and a NEWS bullet.
6. Open the pull request against the branch you started from and fill in the template (Chapter 5 has the checklist).

How to read these notes

1. System setup and Git workflow get you a working fork.
2. Code structure explains what happens when a user calls `bmm()`, and Example models shows two real model files.
3. Adding a new model is the step-by-step guide, ending with a checklist for the pull request.

4. Coding conventions lists what review will check your code against.

The appendix, Reference, collects details you need only when you hit a particular problem, such as within-chain threading or a parameter name that collides with brms.

The rules for contributing

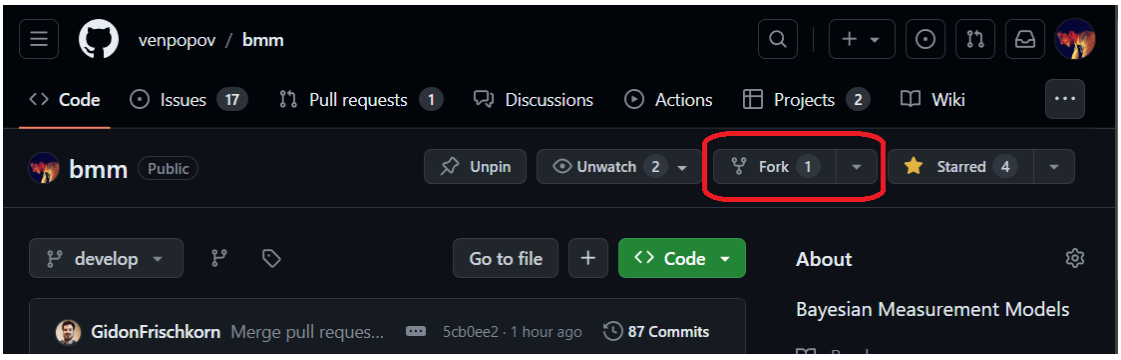
Which branch to target, what a pull request must contain, how many reviews it needs and how model fits for articles are handled are set out in bmm's CONTRIBUTING.md. These notes do not repeat those rules. For a new model, two things matter from the start:

- **Open an issue with the `new_model` template first**, so that you agree with the maintainers that the model fits the package and in which release it will appear.
- **Make the model easy to check.** The reviewer reads your likelihood against its published source and looks at your parameter recovery. Include a short, runnable example with a small simulated dataset, and say in the pull request how long it takes to fit and what output to expect.

1 System setup

1.1 Setting up a local fork of the bmm repository

1. Fork the bmm GitHub repository. This creates a copy of the repository in your own GitHub account.



2. Clone your fork to your local machine, and add the main repository as a remote so that you can keep your branch up to date: `git remote add upstream https://github.com/popov-lab/bmm.git`.

3. Create a branch for your model. Branch from `develop`, or, if the issue for your model is planned for a later release, from that release's integration branch (`dev-x.y.0`); the issue's milestone tells you which. Name the branch after the issue, as `<issue-number>-short-description` (for example `349-lba-model`), so that branch, issue and pull request are easy to line up.

1.2 Prerequisites

The bmm package requires **R >= 4.1.0** (for the native pipe `|>`). It works with both the `rstan` and the `cmdstanr` backend. We recommend `cmdstanr`, which compiles faster and gives better error messages, and the tests that compile Stan code need it:

```
install.packages("cmdstanr", repos = c("https://stan-dev.r-universe.dev", getOption("repos")))
cmdstanr::install_cmdstan()
```

Once bmm is loaded (with `load_all()`, below), `bmm_setup()` checks the C++ toolchain, `cmdstanr` with CmdStan, `rstan`, and the backend `bmm()` will use. It then compiles and samples a small test model and suggests a fix for every check that failed. Run it before your first fit.

1.3 Package development with devtools

Development runs on the `devtools` package, whichever editor you work in. The repository ships a `bmm.Rproj` file, so opening it in RStudio or Positron starts a session in the package root; in any other editor, start R in the repository directory. R Packages is the standard tutorial. The steps you will repeat:

1. Install the development tools and bmm's dependencies once:

```
install.packages(c("devtools", "roxygen2", "testthat", "knitr"))
library(devtools)
install_dev_deps()
```

2. Load the package from your local files whenever you change the code:

```
load_all()  # ctrl+shift+L in RStudio
```

   Do not call `library(bmm)` or source the files, as that loads the installed version, not the one you are working on.

3. Before you open a pull request, regenerate the documentation, run the tests and check the package, in this order:

```
document()  # regenerates man/ and NAMESPACE; never edit those by hand
test()
check()     # 0 errors, 0 warnings; explain any note in the pull request
```

1.4 Running the tests

`test()` runs `tests/testthat/` in parallel. Two kinds of test there skip themselves when your machine cannot run them:

- Tests that compile Stan code need CmdStan. Set `BMM_TEST_STAN=false` in your environment to skip them deliberately.
- Tests that read a stored fit need `tests/testthat/assets/`, which is left out of the built package, so they skip inside `check()`.

Tests that actually sample live in `tests/local/` and never run in `test()` or `check()`. They are not required for a pull request, because a weekly CI job runs them, but you can run them with `Rscript tests/local/run.R`. Run `test(reporter = "slow")` before a pull request as well: the suite has a time budget that CI enforces, and this reporter shows which of your tests are slow. Chapter 5 says which tests a new model needs.

1.5 Untracked scratch work

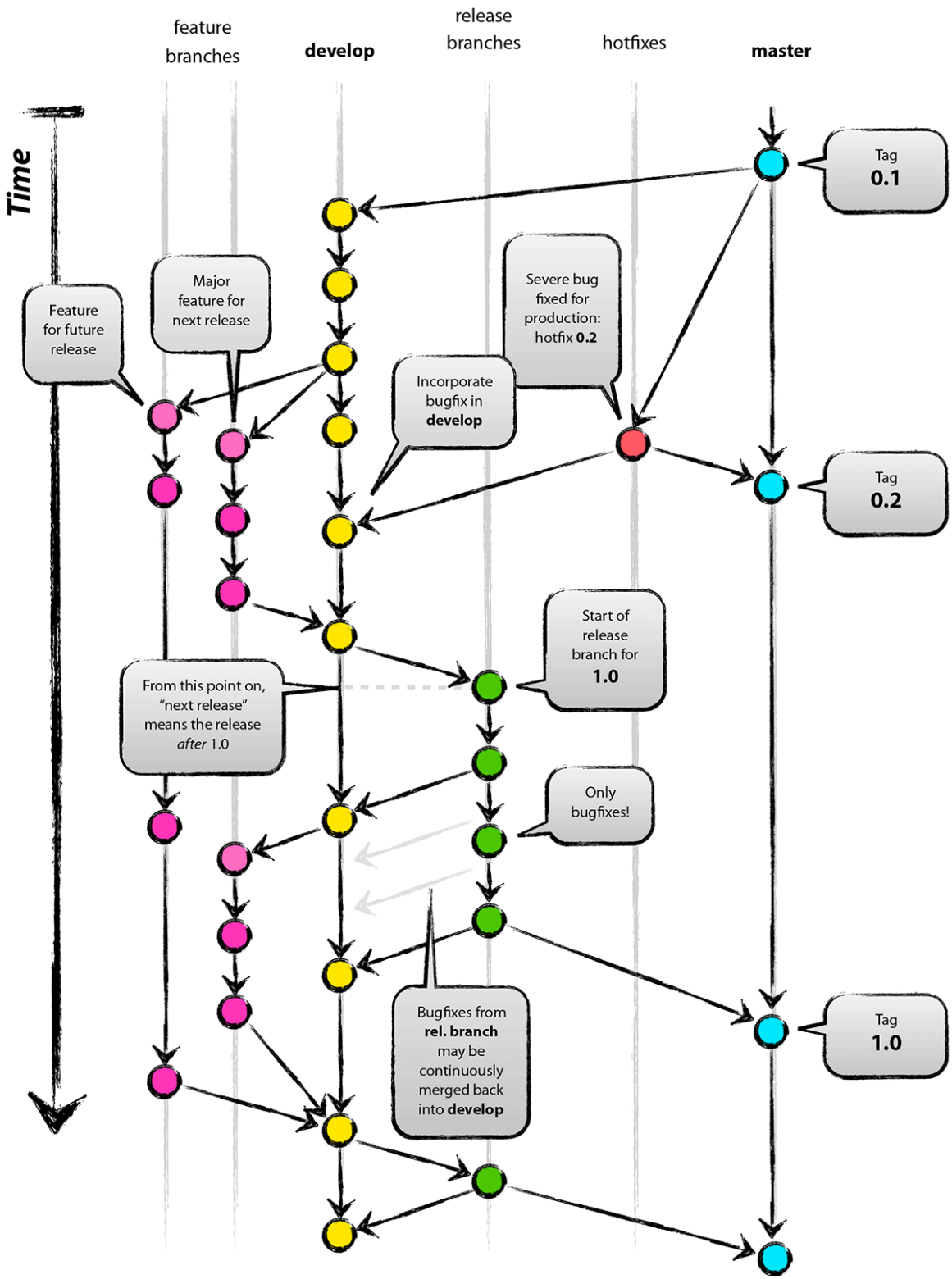
Development produces files that belong on your machine rather than in the package: exploratory scripts, fitted model objects, scratch data. bmm's `.gitignore` excludes a `local/` folder at the root of the repository for this purpose, so anything you put there stays out of your commits. (`tests/local/` is a different folder and is tracked.) Your parameter recovery script can live there, but the reviewer has to see its results; Chapter 5 says how.

2 Git and GitHub workflow

This chapter is a short summary of the commands you need, not a guide to git.

2.1 Branches

bmm follows a git-flow model. `master` holds the latest release and `develop` the next one. Features planned for a later release collect on integration branches named after that release (`dev-x.y.0`), which are merged into `develop` when their release opens. You branch from one of `develop` or `dev-x.y.0` (see Chapter 1) and open your pull request against the same branch.


feature branches
develop
release branches
hotfixes
master
Time
Tag 0.1
Major feature for next release
Severe bug fixed for production: hotfix 0.2
Feature for future release
Incorporate bugfix in develop
Tag 0.2
Start of release branch for 1.0
From this point on, "next release" means the release after 1.0
Only bugfixes!
Bugfixes from rel. branch may be continuously merged back into develop
Tag 1.0

2.2 Git commands

Add your changes, commit them and push them to your fork, from any terminal with the repository as the working directory.

Stage specific files rather than everything with `git add .`, which can pick up files you did not mean to commit, such as fitted model objects:

```
git add R/model_my_model.R inst/stan_chunks/my_model_functions.stan
git commit -m "A short message describing the change"
git push
```

Repeat as often as you like before opening the pull request. Small, self-contained commits are easier to review.

Do not commit fitted models. A CI check fails the pull request if any of its commits adds a file over 1 MiB, even if a later commit removes it again; CONTRIBUTING.md says how fits for articles are handled instead.

2.3 Pull requests

Open the pull request from your fork on GitHub, against the branch you started from. The pull request template asks for what the reviewer needs, with an extra checklist for a new model; fill it in. Review tiers, the recovery evidence a new model needs, and the policy on AI-assisted contributions are in CONTRIBUTING.md.

If you use AI tools, read Chapter 6 before you open the pull request. The ways code generated by a language model typically departs from bmm's style are the ones that chapter lists: validation in internal helpers, base `stop()` and `sprintf()`, a final `return()`, comments that restate the code, and a recovery study that simulates and fits with the same function.

3 BMM code structure

You do not have to edit any of the files described here to add a model: `use_model_template()` (Chapter 5) writes a model file, and the package finds it. This chapter explains what happens to that file when a user fits a model, so that you know which part of it runs when.

3.1 What happens when a user calls `bmm()`

`bmm()` in `R/bmm.R` knows nothing about specific models. It passes the user's formula, data and model to a pipeline of S3 generics, and each model supplies methods for the steps where it differs from the default:

```
bmm(formula, data, model)
|- configure_fit()              R/helpers-model.R
|   |- check_model()            is the model valid, are the user's links allowed
|   |- check_data()             validate and transform the data     <- model method
|   |- check_formula()          one formula per parameter
|   |- configure_model()        brms formula, family, Stan code     <- model method
|   |- configure_prior()        default priors + the user's prior
|   `- create_initfun()         initial values for the sampler
|- brms::brm()                  compile and sample
`- postprocess_brm()            rename, store metadata              <- optional model method
```

`configure_fit()` holds the whole configuration sequence:

The same function serves `stancode()`, `standata()` and `default_prior()`, which stop early through `until`. So the methods you write for `check_data()` and `configure_model()` are also what a user sees when they inspect the Stan code or the default priors of your model without fitting it.

Three details of `bmm()` itself concern model authors:

- **Caching.** With `file_refit = "on_change"`, a cached fit is reused only if the Stan code and data are unchanged. `configure_model()` must therefore generate the same Stan code for the same inputs.

- **Model-frame arguments.** `brms_frame_args()` picks out the `brm()` arguments that change the model frame (`data2`, `knots`, `drop_unused_levels`) and passes them to `configure_prior()` and `create_initfun()`. This is why those methods need `...` in their signature.
- **Threading.** `local_brms_threads()` records whether the user asked for within-chain threading, because a model may need to generate different Stan code for it (Section A.5).

3.2 The `bmmformula` class

Users specify what to estimate with a `bmmformula` (alias `bmf()`): a list of formulas, one per model parameter. Unlike a `brmsformula`, it has no response variable on the left-hand side, because the response is named when the model object is constructed.

```
f <- bmf(kappa ~ 0 + set_size + (1 | id), c ~ 1 + (1 | id), a ~ 1)
f <- bmf(kappa ~ condition, a = 0.5)  # '=' fixes 'a' to 0.5 instead of estimating it
```

A parameter the user does not mention gets an intercept-only formula, with a message. During configuration, the `bmf2bf()` method converts the `bmmformula` to a `brmsformula`. It adds the response as the first formula line and turns each parameter formula into a linear (`brms::lf()`) or non-linear (`brms::nlf()`) component.

3.3 Models

Each model is an S3 class with an internal constructor, `.model_<name>()`, and a user-facing alias, `<name>()`. The parameters, links, fixed parameters, priors and initialization ranges live in a separate list, `.<name>_defaults`, or, for a model with several versions, `.<name>_version_table` (Section A.6). The constructor reads its fields from that list:

```
.my_model_defaults <- list(
  parameters = list(
    par1 = "Parameter 1 = description of parameter 1",
    par2 = "Parameter 2 = description of parameter 2"
  ),
  links = list(par1 = "identity", par2 = "log"),
  fixed_parameters = list(mu = 0),
  priors = list(
    par1 = list(main = "normal(0, 1)", effects = "normal(0, 0.5)", sd = "exponential(1)"),
    par2 = list(main = "normal(0, 0.5)", effects = "normal(0, 0.5)", sd = "exponential(2)")
  ),
```

```
  init_ranges = list(par1 = c(-1, 1), par2 = c(0.5, 1.5))
)

.model_my_model <- function(resp_var1 = NULL, required_arg1 = NULL,
                            links = NULL, call = NULL, ...) {
  out <- structure(
    list(
      resp_vars = nlist(resp_var1),
      other_vars = nlist(required_arg1),
      domain = "",
      task = "",
      name = "",
      citation = character(),
      version = "NA",
      requirements = "",
      parameters = .my_model_defaults[["parameters"]],
      links = .my_model_defaults[["links"]],
      fixed_parameters = .my_model_defaults[["fixed_parameters"]],
      default_priors = .my_model_defaults[["priors"]],
      init_ranges = .my_model_defaults[["init_ranges"]]
    ),
    class = c("bmmodel", "my_model"),
    call = call
  )
  set_links(out, links)
}
```

The fields of the specification:

- `parameters`: the parameters a user can estimate or fix, each with a description. A dummy parameter that exists only for brms stays out (Section A.2).
- `links`: the link function of each parameter, which maps the scale Stan samples on to the scale the parameter is interpreted on. Users can change links through the alias's `links` argument; `set_links()` validates their choice (Section A.1).
- `fixed_parameters`: parameters held at a constant, such as `list(mu = 0)`.
- `priors`: per parameter, a prior for the intercept (`main`), for regression coefficients (`effects`) and for random-effect standard deviations (`sd`); see Section 3.8.
- `init_ranges`: optional `c(lower, upper)` windows for initial values, for parameters whose likelihood is undefined in part of their range (Section A.3).

The metadata fields (`domain`, `task`, `name`, `citation`, `requirements`) fill the help page and the model listings. `domain` decides under which task group `bmm_models()` and the website list the

model, so reuse the string of the models yours belongs with. Tests check that every constructor runs with no arguments and lists its fields in the order above.

The alias validates the user's arguments and calls the constructor. Its help page is generated from the constructor's fields by `model_docs()`:

```
#' @title `r .model_my_model()$name`
#' @name my_model
#' @details `r model_docs(.model_my_model())`
#' @param resp_var1 A description of the response variable
#' @param required_arg1 A description of the required argument
#' @param ... used internally for testing, ignore it
#' @return An object of class `bmmodel`
#' @keywords bmmodel
#' @export
my_model <- function(resp_var1, required_arg1, links = NULL, ...) {
  call <- match.call()
  stop_missing_args()
  .model_my_model(resp_var1 = resp_var1, required_arg1 = required_arg1,
                  links = links, call = call, ...)
}
```

A model with versions adds `version = c("v1", "v2")` to the alias, validates it with `match.arg()` and appends the version to the class. `use_model_template()` generates either variant.

3.4 Class chains

The class vector of a model encodes where it sits in the taxonomy, and it decides which methods run:

```
class = c("bmmodel", <shared class>, <model>, <version>)
```

Task group	Models and class chains
Continuous reproduction	`mixture2p`: `c("bmmodel", "circular", "mixture2p")`; `mixture3p`: `c("bmmodel", "circular", "non_targets", "mixture3p")`; `imm`: `c("bmmodel", "circular", "non_targets", "imm", "imm_full")`; `sdm`: `c("bmmodel", "circular", "sdm", "sdm_simple")`

Task group	Models and class chains
Categorical recall	`m3`: `c("bmmodel", "m3", "m3_ss")`
Processing trees	`mpt`: `c("bmmodel", "mpt")`
Signal detection	`sdt_yn`: `c("bmmodel", "sdt", "sdt_yn")`, likewise `sdt_mafc`, `sdt_ranking`, `sdt_cdp`; `sdt_rating`: `c("bmmodel", "sdt", "sdt_rating", "sdt_rating_standard")`
Choices and response times	`ddm`: `c("bmmodel", "ddm")`; `ezdm`: `c("bmmodel", "ezdm", "ezdm_3par")`; `cswald`: `c("bmmodel", "cswald", "cswald_simple")`

Shared classes such as `"circular"`, `"non_targets"` and `"sdt"` carry methods that several models use: `check_data.circular()` checks that responses are in radians, and `check_formula.sdt()` refuses the stimulus column as a predictor. Dispatch walks the class vector from left to right, so `check_data.bmmodel()` runs first and each method hands on to the next with `NextMethod()`. The model's own method runs last. Chapter 4 follows one such chain through the IMM.

3.5 S3 generics

Each pipeline step is a generic, such as

and a model adds a method for it, `configure_model.my_model()`. That is how new models are added without editing `bmm()`. The generics of the pipeline:

Generic	Signature	Defined in
`check_model`	`(model, data = NULL, formula = NULL)`	`helpers-model.R`
`check_data`	`(model, data, formula)`	`helpers-data.R`
`check_formula`	`(model, data, formula)`	`bmmformula.R`
`bmf2bf`	`(model, formula)`	`bmmformula.R`
`configure_model`	`(model, data, formula)`	`helpers-model.R`
`configure_prior`	`(model, data, formula, user_prior, ...)`	`helpers-prior.R`
`create_initfun`	`(model, data, formula, prior = NULL, ...)`	`helpers-inits.R`
`postprocess_brm`	`(model, fit, ...)`	`helpers-postprocess.R`

Match these signatures exactly, including `...` and the `= NULL` defaults. `R CMD check` compares every method with its generic.

A model writes `check_data()` and `configure_model()` methods almost always, `bmf2bf()` when its response needs wrapping (DDM: `rt | dec(response) ~ 1`) or packs several columns (`vreal()`, `vint()`, `trials()`), and `postprocess_brm()` rarely. `configure_model()` returns `nlist(formula, data, stanvars)`: the `brmsformula` with its family set, the data, and the Stan code for a custom family. A model built on a native brms family, such as `brms::mixture()` for the IMM, needs no `stanvars`.

Further generics let a model adjust behaviour around the pipeline. Each has a default that covers most models; Chapter 5 says when to write a method:

- before and after the fit: `revert_check_data()` (Section 3.7), `settable_links()` (Section A.1), `data_check_findings()`;
- for post-processing: `native_transform()`, `pp_observables()` and `pp_simulate()`.

A few more exist for single model families, such as `generated_parameters()` for MPT or `summary_notes()` for the SDT models. Their source files explain them.

3.6 Passing information from `check_data()` to `configure_model()`

`check_data()` and `configure_model()` are separate methods and cannot share local variables. When `check_data()` computes something that `configure_model()` needs, it stores it as an attribute of the data frame:

```
# in check_data.non_targets() (R/helpers-data.R)
attr(data, "max_set_size") <- max_set_size
attr(data, "lure_idx_vars") <- lure_idx_vars

# later, in configure_model.imm_full() (R/model_imm.R)
max_set_size <- attr(data, "max_set_size")
lure_idx <- attr(data, "lure_idx_vars")
```

Two cautions. The infrastructure uses the same channel for its own bookkeeping (`check_data.bmmodel()` sets `data_name` and `checked`), so not every attribute you find is yours. And a `configure_model()` method can be called directly, without `check_data()`, so the attribute may be missing. `configure_model.sdm()` recomputes the value in that case; if you do not guard, make the failure loud.

3.7 Re-running `check_data()` in `update()`

`update(fit)` without new data refits on the data stored in the fit. That is brms's model frame, which holds only the variables the formula uses and is already the *output* of your `check_data()`.

Running `check_data()` on it again fails for any model whose check adds, renames or packs columns.

So `update()` first calls `revert_check_data(model, data)` to bring the stored frame back into the shape the user's data had. The default returns the data unchanged, which is right for a `check_data()` that only validates. If yours changes columns, write a method in `R/update.R`:

- End with `NextMethod("revert_check_data")`, so that a method on a shared class still runs.
- A column your `check_data()` needs may be missing from the stored frame, because brms keeps only the columns the formula uses. A method that has to rebuild such a column names it in the `"rebuilt"` attribute of the data, and `update()` drops it again after the check. Append to that attribute rather than overwriting it.

`revert_check_data.m3()` and the SDT methods are the examples. The `stored_frame` case your model registers for the tests (Chapter 5) is what checks that this works.

3.8 Default priors

`configure_prior()` reads each parameter's entry in `priors`:

- **`main`** applies to the intercept. If the user drops the intercept (`kappa ~ 0 + condition`), it applies to each level of the factor instead. Keep it first in the list: without an intercept, it is found by position.
- **`effects`** applies to regression coefficients. Without it, brms' defaults apply.
- **`sd`** applies to the random-effect standard deviations. Without it, brms falls back to `student_t(3, 0, 2.5)`, which on a log link lets individual values vary by a factor of 12 around the group value. The shipped models use `exponential()` with a rate chosen by what the parameter measures (Section A.7).

The strings use the format of `brms::set_prior()`. Correlations among random effects get `lkj(2)` without anything in the model definition. For a parameter in both `fixed_parameters` and `parameters`, such as `mu1` in the IMM, the fixed value holds unless the user gives it a formula.

3.9 File organization

File	Contents
`R/bmm.R`	`bmm()`. You do not edit it for a new model.

File	Contents
R/helpers-model.R	configure_fit(), check_model(), configure_model() defaults, use_model_template(), and the central tables a new model adds to: data_column_roles, response_annotations()
R/helpers-data.R, -prior.R, -inits.R, -postprocess.R, -parameters.R	The defaults of the other pipeline steps; also the RT data utilities (flag_contaminant_rts(), ezdm_summary_stats())
R/bmmformula.R	bmmformula()/bmf(), check_formula(), bmf2bf()
R/model_<name>.R	One model: specification, constructor, alias, its methods, and the log_lik, posterior_predict and posterior_epred functions of a custom family. MPT, the largest model, is split over several model_mpt_*.R files.
R/helpers-sdt.R, R/helpers-analysis.R	Code shared by the signal detection models, and their ROC/AUC analyses
R/distributions.R	The d*, r* (and where available p*, q*) functions of each model
R/update.R, R/restructure.R	update() with the revert_check_data() methods; bringing old fits up to date
R/data_check.R, R/bmm_setup.R	bmm_data_check() and bmm_setup()
post-processing files	conditional_effects.R, emmeans.R, pp_check.R, pp_observables.R, ppp.R, summary.R, plot.R, report-extract.R. They work on any fitted model.
R/brms-misc.R	A read-only copy of brms internals. Do not edit it.
inst/stan_chunks/	Stan code for custom families, named <model>_<block>.stan; code shared by several models gets its own file (sdt_dist_funs.stan)

Good model files to read first are R/model_ddm.R (flat specification), R/model_cswald.R (version table) and R/model_sdt_yn.R (aggregated counts).

When you write Stan code, remember that the sampler needs continuous gradients. Non-differentiable operations such as fabs() or fmax() cause divergent transitions; where they are mathematically needed, use a smooth stand-in such as sqrt(x^2 + epsilon) for |x|.

Listing 3.1 R/helpers-model.R

```
configure_fit.bmmformula <- function(formula, data = NULL, model = NULL, prior = NULL,
                                     until = "init", frame_args = list()) {
  user_formula <- formula
  # the user's expression for the data lives in the caller of the generic
  # (bmm() or an extractor). It is set before check_data(), whose methods print
  # it (order_data_query()), and again after, because some methods rebuild the
  # data frame without it
  data_name <- substitute_name(data, envir = parent.frame())
  if (!missing(data) && !is.null(data) && (is.list(data) || is.atomic(data))) {
    attr(data, "data_name") <- data_name
  }
  model <- check_model(model, data, formula)
  data <- check_data(model, data, formula)
  attr(data, "data_name") <- data_name
  formula <- check_formula(model, data, formula)
  config_args <- configure_model(model, data, formula)
  if (until == "model") {
    return(nlist(config_args, model, user_formula))
  }
  prior <- brms::do_call(
    configure_prior, c(list(model, data, config_args$formula, prior), frame_args)
  )
  # the prior decides which parameters exist, so the inits are built from it
  if (until == "init") {
    config_args$init <- brms::do_call(
      create_initfun, c(list(model, data, config_args$formula, prior), frame_args)
    )
  }
  nlist(config_args, prior, model, user_formula)
}
```

Listing 3.2 R/helpers-model.R

```
configure_model <- function(model, data, formula) {
   UseMethod('configure_model')
}
```

4 Example models

This chapter walks through two model files in the package. The IMM is built on a native brms family, `brms::mixture()`, and needs no Stan code of its own; it shows shared classes, `check_data()` dispatch and non-linear formulas. The DDM is a custom family with its own Stan code; it shows a flat specification, a model without a location parameter, and a custom first formula line. Between them they cover what `use_model_template()` asks you to fill in.

4.1 The Interference Measurement Model (IMM)

The IMM lives in `R/model_imm.R`.

4.1.1 Model definition

The fields are described in Chapter 3. What the IMM adds:

- **Versions without a version table.** The IMM predates version tables. It builds the full specification and then removes parameters for the `bsc` and `abc` versions. New models use a version table instead (Section A.6).
- **More response information than one column.** `other_vars` names the non-target feature and distance columns and the set size; with `regex = TRUE` the user can give a pattern instead of the column names.
- **Fixed parameters that identify the model.** `mu2` and `kappa2` are fixed internally. `mu1` is in both `fixed_parameters` and `parameters`, so it is fixed by default and users may estimate it by giving it a formula.
- **Links that cannot be changed.** The IMM writes its links into the mixture family and the non-linear formulas, so `settable_links.imm()` returns `character(0)` and a link a user passes is refused (Section A.1).

4.1.2 `check_data()` and the dispatch chain

The `bsc` and `full` versions need an extra check of the distance columns:

Everything else is inherited. For `imm_full`, with class `c("bmmodel", "circular", "non_targets", "imm", "imm_full")`, the methods run in this order:

1. `check_data.bmmodel()`: the data can be coerced to a data frame and has rows; sets the `data_name` and `checked` attributes.
2. `check_data.circular()`: the response is in radians, wrapped to $(-\pi, \pi]$.
3. `check_data.non_targets()`: checks `nt_features` and `set_size`, creates index variables, and stores `max_set_size` and `lure_idx_vars` as data attributes.
4. No method exists for `imm`, so dispatch passes on.
5. `check_data.imm_full()`: checks `nt_distances`.

General methods run first, so the model's own method can rely on their work: by the time `check_data.imm_full()` runs, `max_set_size` is already an attribute. A new circular model with non-targets inherits all of these checks and only writes its own.

4.1.3 `configure_model()`

The method reads what `check_data()` stored (Section 3.6), builds the formula with `bmf2bf()`, adds the non-linear terms and sets the family. It returns no `stanvars`, because `brms::mixture()` needs no Stan code from bmm.

The three IMM versions differ only in the `theta` expression, so what they share lives in private helpers:

Each version builds `theta_exprs`, one expression per non-target position, and passes it to `.add_imm_mixture_terms()`. This is the pattern to copy when your versions share a formula skeleton: put the skeleton in a helper prefixed with a dot, and let each version supply what differs. Do not start a helper's name with `.model_`, because `model_names()` lists every object with that prefix as a model.

4.1.4 From `bmmformula` to `brmsformula`

`bmf2bf()` builds the first line of the brms formula from the response and adds one component per parameter. Most models use the default:

`bmf2bf.bmmodel()` runs first, because `"bmmodel"` is first in the class vector, and asks the rest of the chain for a first line through `NextMethod()`. If no model method answers, `bmf2bf.default()` returns `NULL` and `bmf2bf.bmmodel()` builds `resp ~ 1` itself. A model that needs a different first line defines its own method, as the DDM below does. Each parameter formula then becomes `brms::nlf()` if one of its predictors is also a parameter and `brms::lf()` otherwise.

4.2 The Diffusion Decision Model (DDM)

The DDM lives in `R/model_ddm.R`.

4.2.1 Model definition

This is the layout `use_model_template()` generates: the specification in `.ddm_defaults`, read field by field by the constructor. The DDM differs from the IMM in three ways:

- **No natural `mu`.** The family gets a dummy `mu`, fixed to 0 through `fixed_parameters` and left out of `parameters` (Section A.2).
- **Two response variables**, `rt` and `response`, which need their own first formula line.
- **`init_ranges`** keep the non-decision time below the fastest response at the start of sampling (Section A.3).

Its class chain is short, `c("bmmodel", "ddm")`, because the response time models share few data checks.

4.2.2 `bmf2bf()`

This produces `rt | dec(response) ~ 1`, where `dec()` tells brms which column codes the decision (0 = lower, 1 = upper boundary).

4.2.3 `configure_model()`

The pieces of a custom family:

- `dpars` lists the dummy `mu` first, with an `identity` link, then the model parameters with the links from `model$links`, so a link the user sets reaches the family.
- `vars = "dec[n]"` passes the decision column to the Stan likelihood. brms connects the family to Stan by name: the family `"ddm"` calls `ddm_lpdf()`, defined in `inst/stan_chunks/ddm_functions.stan`, which `configure_model()` reads and passes as `stanvars`.
- `log_lik`, `posterior_predict` and `posterior_epred` are plain R functions (Chapter 6). The first two are built on `dddm()` and `rddm()` from `R/distributions.R`; `posterior_epred_ddm()` returns the expected response time.

Listing 4.1 `R/model_imm.R`

```
.model_imm <- function(resp_error = NULL, nt_features = NULL, nt_distances = NULL,
                       set_size = NULL, regex = FALSE, version = "full", links = NULL,
                       call = NULL, ...) {
  out <- structure(
    list(
      resp_vars = nlist(resp_error),
      other_vars = nlist(nt_features, nt_distances, set_size),
      domain = "Visual working memory",
      task = "Continuous reproduction",
      name = "Interference measurement model by Oberauer and Lin (2017).",
      citation = glue(
        "Oberauer, K., & Lin, H.-Y. (2017). An interference model \\
          of visual working memory. Psychological Review, 124(1), 21-59. \\
          https://doi.org/10.1037/rev0000044"
      ),
      version = version,
      requirements = glue(
        "- The response vairable should be in radians and \\
          represent the angular error relative to the target
          - The non-target features should be in radians and be \\
          centered relative to the target"
      ),
      parameters = list(
        mu1 = glue(
          "Location parameter of the von Mises distribution for memory \\
            responses (in radians). Fixed internally to 0 by default."
        ),
        kappa = "Concentration parameter of the von Mises distribution",
        a = "General activation of memory items",
        c = "Context activation",
        s = "Spatial similarity gradient"
      ),
      links = list(
        mu1 = "tan_half",
        kappa = "log",
        a = "log",
        c = "log",
        s = "log"
      ),
      fixed_parameters = list(mu1 = 0, mu2 = 0, kappa2 = -100),
      default_priors = list(
        mu1 = list(main = "normal(0, 0.5)", effects = "normal(0, 0.25)", sd = "exponential(4)
        kappa = list(main = "normal(2, 1)", effects = "normal(0, 1)", sd = "exponential(1)")
        a = list(main = "normal(0, 1)", effects = "normal(0, 1)", sd = "exponential(1)"),
        c = list(main = "normal(0, 1)", effects = "normal(0, 1)", sd = "exponential(1)"),
        s = list(main = "normal(0, 1)", effects = "normal(0, 1)", sd = "exponential(1)")
      ),
      # central 50% of the main default prior on the native scale
      init_ranges = list(
        mu1 = c(-0.65, 0.65),
        kappa = c(3.8, 15),
```

Listing 4.2 `R/model_imm.R`

```r
#' @export
check_data.imm_bsc <- function(model, data, formula) {
  data <- .check_data_imm_dist(model, data, formula)
  NextMethod("check_data")
}

#' @export
check_data.imm_full <- function(model, data, formula) {
  data <- .check_data_imm_dist(model, data, formula)
  NextMethod("check_data")
}

.check_data_imm_dist <- function(model, data, formula) {
  nt_distances <- model$other_vars$nt_distances
  max_set_size <- attr(data, "max_set_size")

  stopif(
    !isTRUE(all.equal(length(nt_distances), max_set_size - 1)),
    "The number of columns for non-target distances in the argument \\
    'nt_distances' should equal max(set_size)-1})"
  )

  # replace NA values with 999 so they have 0 effect through the distance formula
  data[, nt_distances][is.na(data[, nt_distances])] <- 999

  stopif(
    any(data[, nt_distances] < 0),
    "All non-target distances to the target need to be postive."
  )
  data
}
```

Listing 4.3 `R/model_imm.R`

```r
configure_model.imm_full <- function(model, data, formula) {
  # retrieve arguments from the data check
  max_set_size <- attr(data, "max_set_size")
  lure_idx <- attr(data, "lure_idx_vars")
  nt_features <- model$other_vars$nt_features
  nt_distances <- model$other_vars$nt_distances

  formula <- .base_imm_formula(model, formula) +
    brms::nlf(theta1 ~ log(exp(c) + exp(a))) +
    brms::nlf(expS ~ exp(s))

  theta_exprs <- glue("{lure_idx} * log(exp(c-expS*{nt_distances}) + exp(a)) + (1 - {lure_idx
  formula <- .add_imm_mixture_terms(formula, max_set_size, lure_idx, nt_features, theta_exprs
  formula$family <- .imm_mixture_family(max_set_size)

  nlist(formula, data)
}
```

Listing 4.4 `R/model_imm.R`

```
.base_imm_formula <- function(model, formula) {
  bmf2bf(model, formula) +
    brms::lf(kappa2 ~ 1) +
    brms::lf(mu2 ~ 1) +
    brms::nlf(kappa1 ~ kappa)
}

.add_imm_mixture_terms <- function(formula, max_set_size, lure_idx, nt_features, theta_exprs)
  kappa_nts <- paste0("kappa", 3:(max_set_size + 1))
  theta_nts <- paste0("theta", 3:(max_set_size + 1))
  mu_nts <- paste0("mu", 3:(max_set_size + 1))

  for (i in 1:(max_set_size - 1)) {
    formula <- formula +
      glue_nlf("{kappa_nts[i]} ~ kappa") +
      glue_nlf("{theta_nts[i]} ~ {theta_exprs[i]}") +
      glue_nlf("{mu_nts[i]} ~ {nt_features[i]}")
  }

  formula
}

.imm_mixture_family <- function(max_set_size) {
  brms::mixture(
    brms::von_mises("tan_half"), brms::von_mises("identity"),
    nmix = c(1, max_set_size),
    order = "none"
  )
}
```

Listing 4.5 `R/bmmformula.R`

```r
# If no model specific methods exist, it will construct a base brms formula from the first re
# We do it this way because most models require just one response variable
#' @export
bmf2bf.bmmodel <- function(model, formula = bmmformula()) {
  brms_formula <- NextMethod("bmf2bf") %||% brms::bf(glue("{model$resp_vars[[1]]} ~ 1"))
  components <- lapply(formula, function(x) if (is_nl(x)) brms::nlf(x) else brms::lf(x))
  Reduce(`+`, components, init = brms_formula)
}

#' @export
bmf2bf.default <- function(model, formula) {
  NULL
}
```

Listing 4.6 `R/model_ddm.R`

```
.ddm_defaults <- list(
  parameters = list(
    drift = "Drift rate = Average rate of evidence accumulation of the decision processes",
    bound = "Boundary separation = Distance between the decision boundaries that need to be r
    ndt   = "Non-decision time = Additional time required beyond the evidence accumulation pr
    zr    = "Relative starting point = Starting point as a proportion of the boundary separat
  ),
  links = list(
    drift = "identity", bound = "log", ndt = "log", zr = "logit"
  ),
  fixed_parameters = list(
    zr = 0,
    mu = 0
  ),
  priors = list(
    drift = list(main = "cauchy(0,1)", effects = "normal(0,0.5)", sd = "exponential(1)"),
    bound = list(main = "normal(0,0.5)", effects = "normal(0,0.5)", sd = "exponential(2)"),
    ndt   = list(main = "normal(-1.5,0.5)", effects = "normal(0,0.3)", sd = "exponential(2)")
    zr    = list(main = "normal(0,0.5)", effects = "normal(0,0.3)", sd = "exponential(2)")
  ),
  init_ranges = list(
    mu = c(-0.1,0.1),
    drift  = c(-1,1),
    bound  = c(1,2),
    ndt    = c(0.01,0.05),
    zr     = c(0.45,0.55)
  )
)

.model_ddm <- function(rt = NULL, response = NULL, links = NULL, call = NULL, ...) {
  out <- structure(
    list(
      resp_vars = nlist(rt, response),
      other_vars = nlist(),
      domain = "Decision Making / Response times",
      task = "Two-Alternative Force Choice RT",
      name = "Diffusion Decision Model",
      citation = glue(
        "Ratcliff, R. (1978). A theory of memory retrieval. Psychological \\
        Review, 85(2), 59-108. https://doi.org/10.1037/0033-295X.85.2.59"
      ),
      version = "NA",
      requirements = glue(
        "- The response time should be in seconds and \\
          represent the time between onset of the target stimulus until the response executi
          - The response should be coded numerically: \\
          0 = lower response, 1 = upper response"
      ),
      parameters = .ddm_defaults[["parameters"]],
      links = .ddm_defaults[["links"]],
      fixed_parameters = .ddm_defaults[["fixed_parameters"]],
```

Listing 4.7 `R/model_ddm.R`

```r
#' @export
bmf2bf.ddm <- function(model, formula) {
  rt <- model$resp_vars$rt
  response <- model$resp_vars$response
  brms::bf(paste0(rt, " | dec(", response, ") ~ 1"))
}
```

Listing 4.8 `R/model_ddm.R`

```r
#' @export
configure_model.ddm <- function(model, data, formula) {
  formula <- bmf2bf(model, formula)

  ddm_family <- function(link_drift, link_bound, link_ndt, link_zr) {
    brms::custom_family(
      "ddm",
      dpars = c("mu", "drift", "bound", "ndt", "zr"),
      links = c("identity", link_drift, link_bound, link_ndt, link_zr),
      lb = c(NA, NA, 0.1, 0, 0),
      ub = c(NA, NA, NA, NA, 1),
      type = "real",
      vars = "dec[n]",
      loop = TRUE,
      log_lik = log_lik_ddm,
      posterior_predict = posterior_predict_ddm,
      posterior_epred = posterior_epred_ddm
    )
  }

  formula$family <- ddm_family(
    link_drift = model$links$drift,
    link_bound = model$links$bound,
    link_ndt = model$links$ndt,
    link_zr = model$links$zr
  )

  sc_path <- system.file("stan_chunks", package = "bmm")
  stan_functions <- read_lines2(paste0(sc_path, "/ddm_functions.stan"))
  stanvars <- brms::stanvar(scode = stan_functions, block = "functions")

  nlist(formula, data, stanvars)
}
```

5 Adding a new model

If you have read Section 3 and Section 4, you should have a pretty good idea of how the `bmm` package functions. Now it's time to add your new model.

You don't have to create the files by hand. `use_model_template()` generates the model file with stubs for every method a model needs, so you can focus on filling them in. The chapter ends with a checklist of everything a new model has to touch *outside* its own file, because the test suite fails on some of those and review catches the rest.

What `use_model_template()` does

Run it from the root of the package. It writes `R/model_<model_name>.R` containing the parameter specification, the model constructor, the user-facing alias, and stub methods for `check_data`, `bmf2bf`, `configure_model` and `postprocess_brm`. With `custom_family = TRUE` it also creates one empty `inst/stan_chunks/<model_name>_<block>.stan` file per entry in `stanvar_blocks`, wires the code that reads them into `configure_model`, and adds empty `log_lik`, `posterior_predict` and `posterior_epred` functions that the family already points to.

The full argument list is on the reference page for `use_model_template()`. Three things about the call are worth knowing:

- **`versions` is the second positional argument.** `use_model_template("gcm", TRUE)` means `versions = TRUE`, not `custom_family = TRUE`. Name the arguments.
- **The stan files are created before the R file is written.** If the call fails partway through, empty `.stan` files are already on disk and you have to remove them by hand before you can call the function again.
- **An unversioned model gets `version = "NA"`** — the literal string, not `NA`.

What the template leaves for you

The generated file follows the package's conventions. Two things in it are placeholders that `check()` will not flag:

- **The prediction functions are empty.** With `custom_family = TRUE`, the family passes `log_lik_<model>()`, `posterior_predict_<model>()` and `posterior_epred_<model>()` to brms, but their bodies are empty, so `loo()`, `pp_check()` and `posterior_epred()` cannot work on your model until you fill them (step 8 below).
- **The `settable_links` stub is commented out.** Uncomment it, so that its tag line reads `#' @exportS3Method`, if `configure_model()` builds links into the family or the formulas; otherwise delete it (step 2 below).

There are no stubs for `check_formula()`, `configure_prior()`, `create_initfun()`, `revert_check_data()` or the optional post-fit methods; add those only if your model needs them.

5.1 Example

Let's add a new model called `gcm`. Assume you have tested the model in Stan and have the Stan code ready. We want a custom family with two Stan blocks, `likelihood` and `functions` (see `?brms::stanvar` for the blocks).

First set up your system and git environment as described in Section 1. Then load the development version with `devtools::load_all()` and run, from the root directory of the `bmm` package:

```
use_model_template("gcm", custom_family = TRUE, stanvar_blocks = c("likelihood", "functions")
```

This creates `R/model_gcm.R` and the empty files `inst/stan_chunks/gcm_likelihood.stan` and `inst/stan_chunks/gcm_functions.stan`, and opens them in your editor. The top of the R file holds the specification and the constructor:

```
.gcm_defaults <- list(
  parameters = list(
    par1 = "Parameter 1 = description of parameter 1",
    par2 = "Parameter 2 = description of parameter 2"
  ),
  links = list(
    par1 = "identity",
    par2 = "log"
  ),
  fixed_parameters = list(
    mu = 0
  ),
  # pick the sd rate by meaning: 1 for sensitivity, strength, concentration
```

```
  # (kappa), mixing weights and identity-linked drift; 2 for criteria,
  # thresholds, boundary, ndt, start point, log-linked drift, diffusion
  # constant, log ratios, log SDs and correlations; 4 for circular bias
  priors = list(
    par1 = list(main = "normal(0, 1)", effects = "normal(0, 0.5)", sd = "exponential(1)"),
    par2 = list(main = "normal(0, 0.5)", effects = "normal(0, 0.5)", sd = "exponential(2)")
  ),
  init_ranges = list(
    par1 = c(-1, 1),
    par2 = c(0.5, 1.5)
  )
)


.model_gcm <- function(resp_var1 = NULL, required_arg1 = NULL, required_arg2 = NULL,
                                    links = NULL, call = NULL, ...) {
  out <- structure(
    list(
      resp_vars = nlist(resp_var1),
      other_vars = nlist(required_arg1, required_arg2),
      domain = "",
      task = "",
      name = "",
      citation = character(),
      version = "NA",
      requirements = "",
      parameters = .gcm_defaults[["parameters"]],
      links = .gcm_defaults[["links"]],
      fixed_parameters = .gcm_defaults[["fixed_parameters"]],
      default_priors = .gcm_defaults[["priors"]],
      init_ranges = .gcm_defaults[["init_ranges"]]
    ),
    class = c("bmmodel", "gcm"),
    call = call
  )
  out <- set_links(out, links)
  out
}

# uncomment if configure_model() builds the links into the family or into
# the non-linear formulas rather than reading them from the list above,
# so that a link set by the user is refused instead of silently ignored:
```

```
# #' @exportS3Method
# settable_links.gcm <- function(model) character(0)
```

Above the specification, a header comment lists the registries outside the file that a new model must update and the format `citation` must follow. One line of that header is out of date in bmm 1.4.1: the stored-frame test case no longer goes into `stored_frame_cases()` in `test-update.R` but into your model's test-case file (see Testing). The other entries, `data_column_roles`, `revert_check_data()`, `response_annotations()` and the `citation` format, are correct. Below it come the user-facing alias `gcm()` with its roxygen block, and the stubs `check_data.gcm()`, `bmf2bf.gcm()`, `configure_model.gcm()` (with a `custom_family()` skeleton and the lines that read your two Stan files), `log_lik_gcm()`, `posterior_predict_gcm()`, `posterior_epred_gcm()` and `postprocess_brm.gcm()`. The `check_data`, `bmf2bf` and `postprocess_brm` stubs carry a header comment saying when you may delete them.

For a complete model written to the current conventions, read `R/model_sdt_yn.R`: a custom family on aggregated counts, with its own `log_lik`, `posterior_predict` and `posterior_epred` functions and a `settable_links` method. Most of its length is documentation.

5.2 Filling in the template

1. **The specification and the constructor.** Most of your work is in `.gcm_defaults`: the parameters (with descriptions), their links, which parameters are fixed and to what value, the default priors, and the initialization ranges. Set default priors (`main`, `effects` and `sd`) for every parameter, informed by knowledge in the field; `.ddm_defaults` in `R/model_ddm.R` is a worked example. Give `init_ranges` only to parameters whose likelihood is undefined in part of their range, such as a non-decision time that must stay below the fastest RT (see Section A.3). Prefer parameter names of more than one letter that do not match a likely data column (see Failure modes to expect).

   In `.model_gcm()`, rename the response and required arguments and fill in the metadata (`domain`, `task`, `name`, `citation`, `requirements`). `citation` holds one reference per element, each on a single line, without a `-` bullet, and ending in `.` or a `https://doi.org/` URL; `model_citation()` hands them to users for their reference lists. Two more things about the constructor are checked by tests: it must run with **no arguments** (the tests call every `.model_*()` bare, which is why every argument defaults to `NULL`), and its fields must stay in the **order the template emits them**. For `domain`, reuse the string of the models your model sits with, so that `bmm_models()` and the website list it under the right task group: `"Visual working memory"`, `"Working Memory (categorical), Categorical Decision Making"`, `"Categorical decision making, memory, and reasoning"` (MPT), `"Perception & Recognition Memory"`

(or "Recognition Memory", which lands in the same group) or "Decision Making / Response times". A new string becomes a group of its own.

For a model with several versions, generate the scaffold with use_model_template("gcm", versions = c("v1", "v2")). You get a .gcm_version_table with one entry per version, a constructor that reads each field from it and appends paste0("gcm_", version) to the class, and an alias with version = c("v1", "v2") validated by match.arg(). Keep the default as a literal c(...): model_versions() reads it to build the per-version documentation tables.

2. **Links.** The constructor passes the user's links through set_links(), which refuses names that are not parameters, repairs one-letter typos with a warning, and warns when a link widens the parameter's range (say identity instead of log). By default every link in your links list can be changed by the user. If configure_model() writes a link into the family, the Stan code or the non-linear formulas instead of reading it from the list, a user's choice would be silently ignored, so declare which links can be set:

```
#' @exportS3Method
settable_links.gcm <- function(model) character(0)   # or c("par1")
```

   Name the method for your model class, not for bmmodel, which comes first in every class vector and would shadow it. A model that substitutes the inverse link into its formulas itself also overrides settable_link_functions(); the multinomial SDT models are the examples.

3. **The user-facing alias.** Rename the required arguments and fill in @examples with a full example. Keep the generated @keywords bmmodel: the pkgdown test needs it and is skipped inside check() (see Chapter 6). Title and details are generated from the constructor.

4. **check_data.gcm()** checks the data and returns it, ending with NextMethod("check_data"). It may compute transformations or store values as data attributes for configure_model (see Section 3.6). If it adds, renames or packs columns, update() needs to undo that before it can re-run it on a fitted model's data; see Section 3.7.

5. **bmf2bf.gcm(), if needed.** You need a custom method if your model has several response variables (DDM: rt | dec(response) ~ 1), uses aggregate data through vreal(), vint() or trials() (EZDM, sdt_yn), or needs other response wrapping. brms reads this formula as the linear model for the family's mu. If your model has no natural mu, suppress it: put mu = 0 in fixed_parameters, add a dummy mu (identity link) to the family's dpars, and write the first formula line yourself (see Section A.2). Leave the dummy mu out of parameters, so that prior_info() and parameter_info() do not report it.

6. **configure_model.gcm()** builds the formula, the family and the stanvars and returns nlist(formula, data, stanvars). It does not return initial values. brms connects

the family to your Stan code by name: a custom family `"gcm"` with `dpars = c("mu", "par1", "par2")` calls a Stan function `gcm_lpdf(y | mu, par1, par2)` (`gcm_lpmf` for an integer response) that you define in `gcm_functions.stan`. The DDM method in Chapter 4 shows the whole pattern, and the brms custom families vignette explains it. Add further Stan files to `inst/stan_chunks/` and read them here. If your Stan code has to differ when the user asks for within-chain threading, see Section A.5.

7. **`postprocess_brm.gcm()`** changes the fitted object (renaming parameters and the like). Delete the section if you need none. If you keep it, also write `revert_postprocess_brm.gcm()`, which undoes the change: `update()` calls it to get the fit back into the shape brms built before refitting. `R/model_sdm.R` has both.

8. **`log_lik`, `posterior_predict` and `posterior_epred`**, for a custom family. These are plain R functions passed to `brms::custom_family()`; the template generates them empty and already passes them to the family, and their contracts are in Chapter 6. They power LOO and WAIC, posterior predictive checks, and `posterior_epred()`/`fitted()`. Build them on `d*()` and `r*()` functions for your model in `R/distributions.R`, which you write first (see Distribution functions). If the mean of your response is not a useful quantity, pass `posterior_epred_undefined("gcm")`, as SDM does, so the user gets an error that points to `native_parameters()`. Models built on a native brms family (`brms::mixture()`, `brms::multinomial()`) get all three from brms.

5.3 Testing

A new model adds two files to `tests/testthat/`:

- **`test-model_gcm.R`** tests your model's own contract: constructor validation, `check_data()`, `configure_model()`, the content of the generated Stan code, and your `d*`/`r*` functions. `tests/testthat/test-model_ddm.R` and `test-model_sdt_yn.R` are examples.
- **`helper-model-gcm.R`** registers your model with the tests that run over every model (default priors, initial values, `update()`, links, posterior predictive checks). It defines one function, `model_cases_gcm()`, which returns small cases those tests use. A `stored_frame`, an `sd_default`, an `init` and a `links` case are required; `check_data`, `observables` and `fixture` cases apply only to some models. Copy `tests/testthat/helper-model-ddm.R` and adapt it. For a model without random-effect quirks or extra observables, it looks like this:

```
model_cases_gcm <- function() {
  model <- gcm("resp", "exemplar", "category")
  data <- toy_gcm_data()  # write a small dataset next to those in helper-model-data.R
  re_formula <- bmf(par1 ~ 1 + (1 | id), par2 ~ 1 + (1 | id))
```

```
  list(
    stored_frame = list(gcm = list(model = model, formula = bmf(par1 ~ 1, par2 ~ 1), dat
    sd_default = list(gcm = list(
      model = model, formula = re_formula, data = data,
      sd = c(par1 = "exponential(1)", par2 = "exponential(2)")
    )),
    init = list(gcm = list(model = model, formula = re_formula, data = data)),
    links = list(gcm = list(model = model, settable = c("par1", "par2")))
  )
}
```

`sd` repeats your default `sd` priors and `settable` what `settable_links()` returns. The header of `helper-model-cases.R` describes the optional cases.

Tests must not sample, so pipeline tests use brms' mock backend. `backend = "mock"` is a `brms::brm()` argument, and whatever you pass as `mock_fit` is returned in place of the fitted Stan object. Everything before the fit (data checks, formula, family, Stan code, priors, initial values) runs for real, which is what you want to test:

```
test_that("gcm model runs without errors", {
  withr::local_options("bmm.silent" = 2)
  skip_on_cran()

  dat <- data.frame(
    resp = rnorm(10),
    exemplar = rep(c("A", "B"), 5),
    category = rep(c(1, 2), each = 5)
  )

  f <- bmmformula(par1 ~ 1, par2 ~ 1)
  mock_fit <- bmm(
    f, dat,
    gcm(resp_var1 = "resp", required_arg1 = "exemplar", required_arg2 = "category"),
    backend = "mock", mock_fit = 1, rename = FALSE
  )
  expect_equal(mock_fit$fit, 1)
})
```

Every argument of the alias without a default has to be supplied, because the alias calls `stop_missing_args()`. Older test files write `mock = 1`, which works only because R partial-matches it to `mock_fit`.

The rest of the rules are numbered R1 to R10 in bmm's AGENTS.md, with the reasoning in ADR 0004. The ones a new model meets:

- Seed with `withr::local_seed()`, never `set.seed()`.
- A test that compiles Stan code opens with `skip_unless_stan_tests()`. A test that takes more than 0.4 s with `NOT_CRAN` unset on your machine opens with `skip_on_cran()`, so that the package's check stays within CRAN's time limit.
- The suite has a time budget. Check your tests with `devtools::test(reporter = "slow")`.
- `tests/testthat/test-conventions.R` checks several of these rules and fails with the rule's number.

`tests/testthat/helper-epred.R` has `epred_vs_predict()` and `expect_epred_by_cell()` for testing a `posterior_epred` function against `posterior_predict`. If your Stan code changes with threading, test the generated code with and without it (Section A.5).

5.3.1 Parameter recovery

A new model needs a parameter recovery study on simulated data. The pull request description shows true against recovered values, as a table or a plot, from a hierarchical design with several subjects whose parameters vary, on data simulated with an independent generator rather than your own `r*` function. Attach the script as a gist or in a collapsed block. Chapter 6 explains why the generator matters, and CONTRIBUTING.md has the requirements.

5.3.2 Distribution functions

A custom family also needs distribution functions in `R/distributions.R`: `d*` and `r*`, optionally `p*` and `q*`. They simulate data, back `log_lik`, `posterior_predict` and `posterior_epred`, and drive the plots. Write them before the prediction functions.

5.4 Before you open the pull request

Everything below lives outside your model file.

The test suite fails without these:

- The two test files above, with all four required cases in your `model_cases_gcm()`.
- An entry in `data_column_roles` (`R/helpers-model.R`) describing in a few words what the column named by each alias argument without a default holds. Use `<model>_<version>` keys where versions differ, and `NA` for an argument that names no column. The *Get started* article builds its model tables from it.
- A `revert_check_data()` method if your `check_data()` adds, renames or packs columns (Section 3.7). The `stored_frame` case is what tests it.

- `@keywords bmmodel` on the alias, and a constructor that runs without arguments and keeps the template's field order.
- At least one reference in `citation`, in the format of step 1.

Review expects these:

- A branch in `response_annotations()` (`R/helpers-model.R`) giving the coding each response variable needs ("radians in [-pi, pi]", "seconds"). `print()` shows it next to each response.
- A `domain` that places the model in the right task group (step 1).
- A parameter recovery study, an example dataset, an article, and a `NEWS.md` bullet.

Optional extension points. The defaults cover most models; write a method only when its condition applies.

Method	Write it when	Example
`pp_observables()`, `pp_simulate()`	the likelihood has more than one observable (RT and choice), so `pp_check()` should check each. Add an `observables` case to your test cases.	`R/model_ddm.R`
`native_transform()`	the map from sampling to native scale is not elementwise `links` plus at most one softmax group	`native_transform.non_targets()`
`data_check_findings()`	users commonly make a data mistake that `check_data()` tolerates; `bmm_data_check()` reports it	`data_check_findings.circular()`
`create_initfun()`, `configure_prior()`	`init_ranges` and the default priors cannot express what the model needs	`create_initfun.m3()`

A model fitted to counts or summary statistics has a few more things to set up (Section A.4).

5.5 Failure modes to expect

- **`configure_prior()` errors when `fixed_parameters` names a parameter the formula never gets.** Every fixed parameter has to be wired into the formula by `configure_model()`. The usual cause is `mu = 0` without a dummy `mu` in the family (Section A.2).

- **Method signatures must match the generics.** `create_initfun()` receives the prior as its fourth argument, and `configure_prior()` methods receive `data2`, `knots` or `drop_unused_levels` through `...` when the user passes them; a method without `...` fails only in that case. `R CMD check` reports both.
- **Short parameter names collide** with Stan parameter names and data columns. Prefer names of more than one letter (Section A.8).

5.6 Post-processing

Most post-fit tools read the model object and need no code from you. `native_parameters()`, `parameter_info()` and `prior_info()` read the specification; `conditional_effects()` and `emmeans()` work through the `bmmfit` class; `posterior_epred()` and `fitted()` call your family's `posterior_epred` function; `pp_check()` checks the response, and further observables if you declared them; `bmm_data_check()` runs your `check_data()` without compiling.

5.7 Add an example dataset

Every new model needs an example dataset that users can load and that the examples use: a new dataset added to the package, one already in it, or one from another package loaded with `data()`. The IMM and the SDT models use datasets shipped with bmm; the `mixture2p` and `mixture3p` articles use `data('bays2009_full', package = 'mixtur')`. R Packages explains how to add one.

5.8 Add an article

Every new model needs an article explaining the model and how to fit it. Create it with `usethis::use_article()`; the existing ones are in `vignettes/articles/`, and the published versions are here. Articles load their fits from cached files, which are not committed; CONTRIBUTING.md says how to hand over the fit. The README, the home page and the *Get started* tables list your model automatically, from its constructor and its `data_column_roles` entry.

6 Coding conventions

This chapter is the reference for how bmm code is written. Your pull request will be judged against these conventions. The Coding style section of bmm's CONTRIBUTING.md and its `AGENTS.md` state them briefly; this chapter gives the reasons.

Read it as conventions, not as a style war. Most of the entries have a reason attached, and the reason is usually that the alternative broke something.

6.1 Errors, warnings and messages

Use the package's own helpers rather than the base functions:

Instead of	Use	Why
`stop()`	`stop2()`	Interpolates with glue, and suppresses the "Error in f(x):" call prefix so the user reads the message, not our internals
`warning()`	`warning2()`	Same
`message()`	`message2()`	Same, and respects `options(bmm.silent)`
`if (cond) stop2(...)`	`stopif(cond, ...)`	One line, and reads as the condition it guards
`stopifnot()`	`stopif(cond, ...)`	`stopifnot()` messages name the failed expression, not the user's argument
`if (cond) warning2(...)`	`warnif(cond, ...)`	Same
`sprintf()` in a message	`glue::glue()`	Named interpolation, no positional arguments to miscount
`paste(x, collapse = ", ")`	`collapse_comma(x)`	Also quotes each element

The base functions survive almost only inside the definitions of the helpers themselves and in `R/brms-misc.R`, which is vendored code: a copy of brms internals kept inside bmm rather than called through `brms::`, and marked read-only. `sprintf()` is still the right tool for formatting numbers (`"%.2f"`); the rule is about building messages.

Two of the reasons deserve spelling out, because they are not stylistic.

`message2()` checks `getOption("bmm.silent", 1)` and prints nothing at silence level 2 or above. Tests set `withr::local_options("bmm.silent" = 2)` to keep output clean. A base `message()` call ignores that and will make somebody's test noisy.

`stopif()` and `warnif()` pass `env.frame = -2` down to `stop2()`. That number counts frames back up the call stack: `-1` would interpolate in `stopif()`'s own frame, where your variables do not exist, so `-2` reaches past it into the function that called `stopif()`. This is what lets the message interpolate variables from *your* function. So this works and needs no `glue()` call of its own:

```
stopif(
  !version %in% c("custom", "cs", "ss"),
  'Unknown version: {version}. It should be one of "ss", "cs" or "custom"'
)
```

Write messages as one glue string. `stop2()` handles multi-line strings with trailing `\\` continuations, so a long message does not need `paste0()`.

6.2 Dependencies

Package code is **base R plus the native pipe `|>`**. bmm requires R >= 4.1.0, so `|>` is always available and `%>%` is not used in package code.

The Imports in `DESCRIPTION` are a short list, and adding to it needs a reason the reviewer will accept. **dplyr, tidyr, ggplot2 and stringr are `Suggests`, not Imports** — dplyr and tidyr became optional in 1.2.0, and magrittr was removed in 1.3.0. If you have seen bmm guidance that prescribes dplyr idiom for package code, it is out of date. Tidyverse packages belong in articles and tests, where they are already available.

A function from a `Suggests` package must be guarded, or the package breaks for anyone who installed it without that suggestion. The pattern is:

```
stopif(
  !requireNamespace("tidyr", quietly = TRUE),
  'The "tidyr" package is required for this functionality'
)
data <- tidyr::gather(data, non_target_name, non_target_value, eval(nt_features))
```

as in `R/helpers-data.R`. Note the same function opens with

```
y <- y_nt <- non_target_name <- non_target_value <- NULL
```

which exists to silence `R CMD check`'s "no visible binding for global variable" notes from the non-standard evaluation in the `gather()` call. If you write code with bare column names, you need this line.

Always namespace explicitly: `brms::stanvar()`, `stats::complete.cases()`, `methods::formalArgs()`. bmm imports few functions directly, and an unqualified call to an Imports package is the kind of thing that works on your machine and fails in `R CMD check`.

6.3 Code shape

Return the last expression; use `return()` only to leave early. Guard clauses are the endorsed use:

```
x <- try_read_bmmfit(file, file_refit)
if (!is.null(x)) {
  return(x)
}
```

Do not write `return(out)` as the last line of a function.

No `set.seed()` inside a function, and no `seed` argument. A function that sets the seed silently rewrites the caller's random state. `set.seed()` appears in bmm only in `@examples` blocks in the roxygen comments — the `#'` comments above each function from which `devtools::document()` generates the help pages — where it is the right thing. If you need reproducibility inside a function, take the draws you were given; in a test, use `withr::local_seed()`.

snake_case for functions and variables. Private helpers are prefixed with a dot (`.base_imm_formula`, `.imm_mixture_family`). One reservation: a private helper must **not** be named `.model_*`, because `model_names()` lists every object matching `^\.model_` as a model.

Comments say why, not what. `# construct the family` above a `brms::custom_family()` call adds nothing. The comment in `configure_model.sdm()` —`# note - c has a log link, but I've coded it manually for computational efficiency` — is worth its line, and so is every comment that records a constraint somebody would otherwise remove.

6.4 S3 conventions

Dispatch runs general to specific. A model's class vector starts with `"bmmodel"` (the class chains are listed in Section 3.4), so `check_data.bmmodel` runs *before* `check_data.sdm`, not after. This is the opposite of the intuition most people bring, and it has one sharp corollary: **a fall-through fallback belongs on `.default`, never on `.bmmodel`.** A method defined on `.bmmodel` runs first and shadows nothing, but a *fallback* placed there would be reached before the model-specific method had a chance to run. `bmf2bf` is the worked example — `bmf2bf.default` returns `NULL` and `bmf2bf.bmmodel` supplies the fallback after `NextMethod()` has come back empty (see Chapter 4).

Every `check_data` method ends with `NextMethod("check_data")`. Each method does its own model-specific validation and then hands on down the chain. The only method without it is `check_data.default`, which is the end of the chain and returns the data. Forget it in a method on a shared class (`circular`, `non_targets`, `sdt`) and every more specific method below it never runs, so a model's own checks are silently skipped. In a model's own method the omission does no harm today, but the next shared class added above it would break.

Match the generic's signature exactly, including `...` where the generic has one and the defaults on `check_model(model, data = NULL, formula = NULL)`. See the generics table in Chapter 3.

6.5 Where to validate arguments

Validate in the **exported constructor only** — the user-facing alias. `.model_*()`, `configure_model.*()` and private helpers should not, because their error messages name arguments the user never typed. The alias is the one place where the argument the message refers to is the argument the user actually passed:

Listing 6.1 `R/model_m3.R`

```r
m3 <- function(resp_cats, num_options, choice_rule = "softmax", version = "custom", ...) {
  call <- match.call()
  stop_missing_args() # errors naming any argument with no default that the user omitted
  stopif(
    length(num_options) != length(resp_cats),
    "The option variables should have the same length as the response variables."
  )
  .model_m3(...)
}
```

`check_data.*()` is the exception, and not really an exception: it validates properties of the *data*, which the constructor cannot see. "Your `nt_distances` columns contain negative values" is a data message and belongs there.

6.6 `log_lik`, `posterior_predict` and `posterior_epred`

These are **plain functions, not S3 methods.** Do not name them `log_lik.mymodel` — that would register them against brms' `log_lik` generic, which is not what you want. Name them `log_lik_<family_name>` and pass them to `brms::custom_family()`:

```
sdm_simple <- brms::custom_family(
  name = "sdm_simple",
  dpars = c("mu", "c", "kappa"),   # distributional parameters
  ...
  log_lik = log_lik_sdm_simple,
  posterior_predict = posterior_predict_sdm_simple,
  posterior_epred = posterior_epred_sdm_simple
)
```

brms checks the first arguments when you build the family: `log_lik` must start with `(i, prep)` and `posterior_predict` with `(i, prep, ...)`.

Both take `i`, the index of one observation, and `prep`, an object brms assembles from the fitted model and hands to you. `prep` holds the posterior draws and the data, and you pull values out of it rather than computing them: `brms::get_dpar()` for a parameter, `prep$data$Y` for the response.

The contract is easy to get wrong in one specific way. `brms::get_dpar(prep, "par", i = i)` returns a **vector with one value per posterior draw**, not a scalar, and your function must return a vector of that same length. The response is a scalar, `prep$data$Y[i]`:

Listing 6.2 `R/model_sdm.R`

```
log_lik_sdm_simple <- function(i, prep) {
  mu <- brms::get_dpar(prep, "mu", i = i)
  c <- brms::get_dpar(prep, "c", i = i)
  kappa <- brms::get_dpar(prep, "kappa", i = i)
  y <- prep$data$Y[i]
  dsdm(y, mu, c, kappa, log = T)
}
```

A density function that collapses its arguments (an `if` on a vector, a `sum()`) will return a scalar here, and brms will recycle it across all draws without complaint. The posterior predictive checks then look suspiciously clean. Check `length()` of what you return.

`posterior_epred_<family_name>` has a different contract: it takes only `prep`, is called once for all observations, and returns a **draws × observations matrix** of expected responses. `prep$dpars` then holds one draws × observations matrix per parameter, and data columns have to be lined up with them. Two helpers in `R/helpers-postprocess.R` do the bookkeeping: `.epred_data()` repeats a data column per draw, and `.epred_matrix()` shapes the result:

Listing 6.3 `R/model_sdt_yn.R`

```
posterior_epred_sdt_yn <- function(prep) {
  stimulus <- .epred_data(prep$data$vint1, prep)
  dist <- .sdt_dist_names[prep$data$vint2[1]]
  criterion <- .sdt_criterion_dpar(prep, prep$dpars$d, prep$dpars$sdratio)
  p_old <- with(prep$dpars, exp(.sdt_log_p_old(.sdt_eta(d, criterion, stimulus, sdratio), dis
  .epred_matrix(.epred_data(prep$data$trials, prep) * p_old, prep)
}
```

Pass it to `custom_family()` like the other two. A custom family without one fails in `posterior_epred()` with "object 'posterior_epred_' not found", because brms looks for the function in the global environment. If the expected response of your model is not meaningful, pass `posterior_epred_undefined("<model>")`, which fails with a message pointing to `native_parameters()`.

For a custom family on aggregated data, `loop` and `vars` decide what the Stan likelihood can index; see Section A.4. The names you give a family's parameters have traps of their own (Section A.8).

6.7 Documentation and the checks that enforce it

`NAMESPACE` and everything in `man/` are **generated**. Run `devtools::document()`; never hand-edit them.

Exported functions need `@returns` and `@examples`. Two further requirements are enforced by tests rather than by review:

`@keywords`. The package website is built by pkgdown, which needs every public help topic to appear somewhere in its reference index, and `_pkgdown.yml` is where that index is declared. `tests/testthat/test-pkgdown.R` reads every `man/*.Rd` file, drops the ones with `\keyword{internal}`, and fails if any remaining topic is not covered by the reference section of `_pkgdown.yml`. Coverage can be by name or through a `has_keyword()` selector, and

`_pkgdown.yml` has `has_keyword("bmmodel")`. So a new model's user-facing alias needs `#' @keywords bmmodel`, which every model alias carries and which `use_model_template()` generates. Keep it when you edit the roxygen block. An exported function that is not a model gets `@keywords internal` or a place in `_pkgdown.yml`.

A NEWS bullet for every user-visible change, under the development heading at the top of `NEWS.md` (on a `dev-x.y.0` branch, under that release's heading), with the issue or pull request number in parentheses. Keep it short: what changes for the user and what they should do, not how the fix works.

6.8 Tests

Test files mirror source file names: `R/model_ddm.R` is tested by `tests/testthat/test-model_ddm.R`. Some files cover a theme instead (`test-default-priors.R`, `test-pkgdown.R`). Tests use `testthat` edition 3, the mock backend instead of sampling, and `withr::local_options("bmm.silent" = 2)` so that `message2()` output does not clutter the run. The rules that apply to every test are numbered R1 to R10 in `AGENTS.md`; Chapter 5 lists the ones a new model meets.

6.9 Parameter recovery: the circularity trap

What a recovery study for a new model has to show is set out in CONTRIBUTING.md: true against recovered values, a hierarchical design with several subjects, and data from an independent generator. The last point is the one that is easy to get wrong.

Do not simulate and fit with the same distribution function. If you generate data with your own `rmymodel()` and fit a likelihood built from your own `dmymodel()`, the study reports clean recovery whether or not either function is correct, because any error shared by both cancels. Use an independent generator: `rtdists::rdiffusion()` for diffusion-type models (`rtdists` is already in Imports), a published reference implementation, or a direct simulation of the process the model describes. A recovery study that cannot fail is not evidence.

A Reference

This appendix collects details you need only in particular situations. Each section starts by saying when that is.

A.1 Links

You need this when your model's links cannot all be changed by the user, or when a user reports that a link they set had no effect.

bmm implements the links `"identity"`, `"log"`, `"softplus"`, `"log1p"`, `"logm1"`, `"inverse"`, `"sqrt"`, `"logit"`, `"probit"`, `"cloglog"` and `"tan_half"`. A model may also declare `"loglog"` or `"softmax"` as its own default, but users cannot ask for them. `"softplus"` is an alternative to `"log"` for positive parameters: it grows linearly rather than exponentially, so predictor effects are additive on the natural scale and large values do not overflow.

The constructor assigns the user's links with `set_links()`. It refuses a name that is not a parameter (a one-letter typo is repaired, with a warning), refuses a link the model cannot apply, and warns when the new link admits values the default excludes, such as `identity` for a positive parameter. `check_model()` runs the same validation on links assigned after construction.

By default every link can be changed. A model that writes its links into the family, the Stan code or the non-linear formulas, instead of reading them from `model$links` at fit time, must say so with a `settable_links()` method that returns `character(0)` or the subset that works. Otherwise a user's link would show in `print()` and in the priors while the sampler ignored it. SDM, the mixture models and the IMM return `character(0)`; the SDT models return the parameters whose links reach the likelihood. A model that substitutes the inverse link into its formulas itself also overrides `settable_link_functions()`, as M3, MPT and the multinomial SDT models do.

A.2 Models without a location parameter

You need this when your model has no natural `mu`, as is typical for response time models.

brms requires every custom family to have a `mu` parameter. DDM, EZDM and cswald have none, and suppress it rather than promote one of their parameters to it:

1. The family's `dpars` include a dummy `mu` with an `identity` link.
2. `fixed_parameters` contains `mu = 0`.
3. `mu` is left out of `parameters`, so `prior_info()` and `parameter_info()` do not report it, and users do not give it a formula.
4. The model's `bmf2bf()` method writes the first formula line itself, such as `rt | dec(response) ~ 1` for the DDM.

If you add `mu = 0` to `fixed_parameters` but forget the dummy `mu` in `dpars`, `configure_prior()` errors, because every fixed parameter has to exist in the formula.

A.3 Initial values

You need this when chains fail at initialization or diverge immediately, typically because the likelihood is undefined for part of a parameter's range.

brms' default initialization draws every parameter from a wide symmetric range on the sampling scale. For a non-decision time that must stay below the fastest observed RT, that often starts chains where the likelihood is undefined. `init_ranges` gives such parameters a narrow window on the native scale:

```
init_ranges = list(
  mu    = c(-0.1, 0.1),
  drift = c(-1, 1),
  bound = c(1, 2),
  ndt   = c(0.01, 0.05)
)
```

`create_initfun.default()` in `R/helpers-inits.R` reads the parameter dimensions from the Stan code and builds an initialization function:

1. Random effects start tight: standard deviations between 0.05 and 0.1, standardized effects near zero, correlation matrices at the identity.
2. Population-level parameters with an `init_ranges` entry start inside that range. The range is transformed through the parameter's link (for softmax mixing weights it is already on the sampling scale), so it stays valid when the user changes the link. With predictors, the coefficients are solved so that each design cell starts inside the range.
3. Everything else gets brms' default draw.

`create_initfun()` maps Stan parameter names back to model parameters with the regular expression `(^|_)param(_|$)`, taking the longest match. A parameter called `s` therefore matches `b_s_Intercept` but not `b_sim_Intercept`. A name that appears as a whole underscore-delimited token elsewhere in the Stan parameter names is not safe.

A.4 Aggregated data and `vars`

You need this when your model is fitted to counts or summary statistics instead of one row per trial.

brms has special response terms that pass further columns to the Stan likelihood: `vreal()` for real values, `vint()` for integers and `trials()` for a trial count. The EZDM's `bmf2bf()` method builds `mean_rt | vreal(var_rt) + vint(n_upper) + trials(n_trials) ~ 1`, which makes the columns available in Stan as `vreal1[n]`, `vint1[n]` and `trials[n]`. The family names them in `vars`:

```
brms::custom_family(..., loop = TRUE, vars = c("vreal1[n]", "vint1[n]", "trials[n]"))
```

`loop` decides the shape of the generated Stan code. With `loop = TRUE`, brms emits one likelihood call per observation; with `loop = FALSE`, one vectorized call. You may index `vars` with `[n]` only in a looped likelihood; otherwise brms stops with "Invalid use of index '[n]' in an unlooped custom likelihood". SDM uses `loop = FALSE` and no `vars`.

Add a model fitted to counts to `uses_aggregate_data()` in `R/data_check.R` (currently `m3`, `ezdm` and `mpt`), so that `bmm_data_check()` does not report too few observations per cell.

A.5 Within-chain threading

You need this when your Stan code injects data or likelihood code through `stanvar()`, or your custom family uses `vars` with `loop = FALSE`.

Passing `threads` to `bmm()` asks brms to split each chain's likelihood across cores. brms slices the response and calls the likelihood inside `partial_log_lik`, where the row bounds `start` and `end` exist. Two things it does *not* slice:

- **A custom family's `vars`.** brms pastes them in as written, so a vectorized likelihood would receive the whole vector next to a slice of the response. cswald therefore passes `vars = "dec[start:end]"` when threading and `"dec"` otherwise.
- **Stan code injected through `stanvar()`**, which has to be written differently depending on whether it ends up inside `partial_log_lik` or in the model block.

Both decisions rest on `brms_slices_likelihood()` in `R/utils.R`. Call it in `configure_model()` rather than reading the `brms.threads` option yourself: it normalizes the option as brms does and is `FALSE` for `threading(force = TRUE)`, where brms compiles with threading support but emits unsliced code. SDM, cswald and `sdt_mafc` use it.

SDM is the fullest example. `inst/stan_chunks/` holds `sdm_simple_likelihood.stan` and `sdm_simple_likelihood_threaded.stan`, and `configure_model.sdm()` reads one of them:

```
likelihood_file <- if (brms_slices_likelihood()) {
  "sdm_simple_likelihood_threaded.stan"
} else {
  "sdm_simple_likelihood.stan"
}
```

Data the threaded likelihood needs must also be declared in the partial-log-likelihood signature through the `pll_args` argument of `stanvar()`. SDM passes `"data matrix COSN"` and, through its helper `sdm_stanvar_int_array()`, the run-index arrays. Without that, the variable is out of scope inside `partial_log_lik` and the model does not compile.

Test the generated Stan code with no `threads`, with `threads = 2` and with `threading(2, force = TRUE)`, as `test-model_sdm-stancode.R` does.

A.6 Version tables

You need this when your model has several versions, or when you read an older model file and its structure differs from what the template generates.

`use_model_template("mymodel", versions = c("v1", "v2"))` generates a version table: the specification of each version in one lookup list, which the constructor indexes with `version`. EZDM, cswald and M3 use it:

```
parameters = .ezdm_version_table[[version]][["parameters"]],
```

Keep the default of the alias's `version` argument a literal `c(...)`: `model_versions()` reads it to build the per-version documentation.

Two departures from this shape exist in the package:

- **M3 nests one level deeper.** Its `links` and `priors` are indexed by the choice rule before the parameter (`.m3_version_table[[version]][["links"]][[choice_rule]]`), `fixed_parameters` is computed in the constructor, and the default version `"custom"` is not in the table at all, because the user supplies the specification. Copy this only if your model has a second axis of variation.

Listing A.1 `R/model_ezdm.R`

```
.ezdm_version_table <- list(
  "3par" = list(
    parameters = list(...),
    links = list(...),
    fixed_parameters = list(...),
    priors = list(...),
    init_ranges = list(...)
  ),
  "4par" = list(
    parameters = list(...),
    links = list(...),
    ...
  )
)
```

- **Some models write their specification inline.** `sdm`, `mixture2p`, `mixture3p`, `imm` and most SDT models have no defaults list or version table, because they predate it. New models use the separated specification.

A.7 Choosing the `sd` prior rate

You need this when you set the `sd` default priors of a new model.

The shipped models use `exponential()` on the link scale, with the rate chosen by what the parameter measures:

Rate	Parameters
1	sensitivity, memory strength, concentration (`kappa`), mixing weights, identity-linked drift
2	criteria and thresholds, boundary, non-decision time, start point, log-linked drift, diffusion constant, log ratios, log SDs, correlations
4	circular bias

The template repeats this table as a comment above the priors. Every model declares an `sd` prior, except the custom version of M3, which has no default priors at all.

A.8 Naming traps

You need this when brms or Stan fails with an error that does not point at the cause, or when a parameter gets a prior you did not set.

A custom family's `dpars` may not contain `_` or `.`, and may not end with a digit. `brms::custom_family()` checks both, so neither `delta_1` nor `delta1` is a legal name. The digits on IMM parameters (`kappa1`, `theta3`) are generated by `brms::mixture()`, and parameters introduced through `nlf()` are not subject to this check.

Reusing a brms dpar name inherits its default prior. brms attaches default priors by name to `mu`, `sigma`, `shape`, `nu`, `phi`, `kappa`, `beta`, `zi`, `hu`, `zoi`, `coi`, `bs`, `ndt`, `bias`, `quantile`, `xi`, `alpha`, `disc` and `theta`, and strips trailing digits first, so `kappa1` is treated as `kappa`. bmm reuses several of these names and sets its own defaults; the trap is a reused name you forget to give a default prior.

A dpar must not be uniquely prefixed by `thres`. brms reads `p$thres` when it builds a custom likelihood call, and `$` partial-matches on lists, so a dpar named `threshold` is passed where ordinal thresholds belong. The result is Stan code that does not parse.

Stan probability-function suffixes need the bar syntax. A function whose name ends in `_lpdf`, `_lupdf`, `_lpmf`, `_lupmf`, `_cdf`, `_lcdf` or `_lccdf` must be called with a vertical bar between its first two arguments, as `swald_lpdf(rt | drift, bound, ndt, s)`. If your helper is not really a distribution function, give it another name.

An `nlf()` right-hand side is pasted together as text, so operator precedence is yours to manage. The IMM parenthesizes every sub-expression it interpolates: `"{lure_idx} * (-expS*{nt_distances} + c) + (1 - {lure_idx}) * (-100)"`.

Short parameter names collide. Besides the initial-value matching above (Section A.3), bmm treats a formula component as non-linear when one of its predictors is also a parameter. When the same name is also a data column, `check_formula()` warns, because the choice changes the likelihood without the user seeing it. One-letter parameters collide with columns such as a condition coded `c`.